

Αναφορά Προόδου Διπλωματικής Εργασίας

Ανίχνευση και Αντιμετώπιση Επιθέσεων σε Δίκτυα
Οριζόμενα από Λογισμικό (SDN)

Φοιτήτρια: Ζαφειρώ Μπάρδα
Ημερομηνία: 7 Ιουλίου 2026
Αποθετήριο: github.com/ZafeiroBarda/ptyxiaki

Κατάσταση έργου: πλήρως λειτουργικό — 70/70 αυτοματοποιημένα tests επιτυχή, συνεχής
ενσωμάτωση (CI) πράσινη

1. Σύνοψη — Τι έχει γίνει μέχρι τώρα

Η εργασία υλοποιεί ένα ολοκληρωμένο σύστημα ανίχνευσης και αυτόματης αντιμετώπισης επιθέσεων σε δίκτυα SDN, γραμμένο εξ ολοκλήρου σε Python. Το έργο δεν είναι ένα απλό μοντέλο μηχανικής μάθησης· είναι ένα πλήρες πειραματικό σύστημα που καλύπτει δύο συμπληρωματικές προσεγγίσεις και επικυρώνεται τόσο σε δημόσιο dataset όσο και σε ζωντανή προσομοίωση δικτύου.

Οι δύο προσεγγίσεις:

- **Μέθοδος Β — Offline ανίχνευση με ML:** Εκπαίδευση και σύγκριση πολλών επιβλεπόμενων μοντέλων (Random Forest, SVM, νευρωνικά δίκτυα, Decision Tree, KNN, Logistic Regression, Deep MLP, 1D-CNN) σε dataset τύπου InSDN για offline ανίχνευση.
- **Μέθοδος Α — Live ανίχνευση και αντιμετώπιση:** Πλήρης αρχιτεκτονική SDN σε επίπεδο εφαρμογής (Control Plane + Defense Plane σε Flask, Isolation Forest ως μη-επιβλεπόμενος ανιχνευτής, dashboard σε πραγματικό χρόνο) που ανιχνεύει ανωμαλίες και εγκαθιστά αυτόματα κανόνες DROP για μπλοκάρισμα της επίθεσης σε υπο-δευτερόλεπτο χρόνο.

Η εργασία συνοδεύεται από πλήρη διπλωματική εργασία 102 σελίδων (~20.400 λέξεις, 8 κεφάλαια συν παραρτήματα), αυτοματοποιημένα tests, συνεχή ενσωμάτωση (GitHub Actions) και επαληθευμένα πειραματικά αποτελέσματα.

Πρόσφατη εργασία (τελευταία μέρα): εντοπίστηκαν και διορθώθηκαν αστοχίες στο pipeline συνεχούς ενσωμάτωσης· και οι 70 αυτόματοι έλεγχοι περνούν πλέον καθαρά και η ροή CI στο GitHub είναι πράσινη.

Πίνακας κατάστασης συνιστωσών

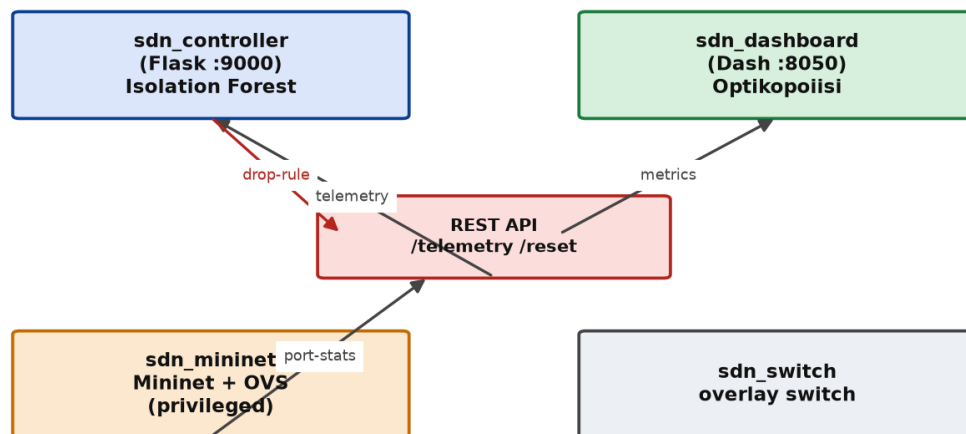
Συνιστώσα	Περιγραφή	Κατάσταση
Offline ML pipeline	8 μοντέλα, εκπαίδευση + αξιολόγηση	Ολοκληρωμένο
Επικύρωση full InSDN	6 μοντέλα + IF σε 343.516 πραγμ. ροές	Ολοκληρωμένο
Isolation Forest (live)	Μη-επιβλεπόμενη ανίχνευση ανωμαλιών	Ολοκληρωμένο
SDN Controller (Flask)	Control + Defense Plane, REST API	Ολοκληρωμένο
Dashboard πραγμ. χρόνου	Οπτικοποίηση τοπολογίας & επιθέσεων	Ολοκληρωμένο
Σενάρια επιθέσεων	5 σενάρια (scan, DDoS, exfiltration κ.ά.)	Ολοκληρωμένο
Ασφάλεια Control Plane	Token auth, rate limiting, injection log	Ολοκληρωμένο
Adversarial robustness	Επιθέσεις παραβίασης + robust training	Ολοκληρωμένο
Αυτόματα tests (pytest)	70 tests σε 7 αρχεία	70/70 επιτυχή
Συνεχής ενσωμάτωση (CI)	GitHub Actions: 3 jobs	Πράσινη
Διπλωματική (docx)	102 σελίδες, 8 κεφάλαια	Πλήρες προσχέδιο

2. Αρχιτεκτονική του συστήματος

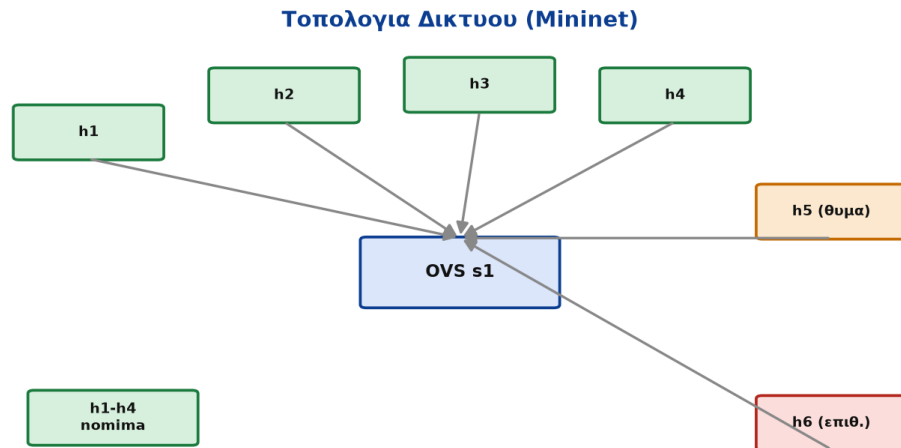
Το σύστημα ακολουθεί την κλασική διαστρωμάτωση των δικτύων SDN σε επίπεδα (planes), υλοποιημένη ως microservices που επικοινωνούν μέσω REST:

- **Control Plane:** ο κεντρικός controller (Flask) διατηρεί τη σφαιρική εικόνα του δικτύου (γράφος κόμβων και ενεργών ροών) και τον πίνακα ροών (flow table).
- **Defense / Intelligence Plane:** ο Isolation Forest αναλύει 8 συγκεντρωτικά χαρακτηριστικά ανά ροή· όταν εντοπίσει ανωμαλία, ο controller εγκαθιστά κανόνα DROP και μπλοκάρει την πηγή αυτόματα.
- **Data Plane:** οι switches στέλνουν τηλεμετρία (στατιστικά ροών) στον controller μέσω του endpoint /telemetry, προσομοιώνοντας τη λογική του OpenFlow.
- **Management Plane:** dashboard σε πραγματικό χρόνο (Dash/Cytoscape) που οπτικοποιεί την τοπολογία, τον ρυθμό πακέτων και τα μπλοκαρισμένα hosts.

Αρχιτεκτονική Συστήματος (Docker)



Σχήμα 1. Αρχιτεκτονική του συστήματος — τα επίπεδα SDN και η ροή δεδομένων.



Σχήμα 2. Τοπολογία δικτύου προσομοίωσης (6 hosts, 1 switch).

Ασφάλεια του Control Plane

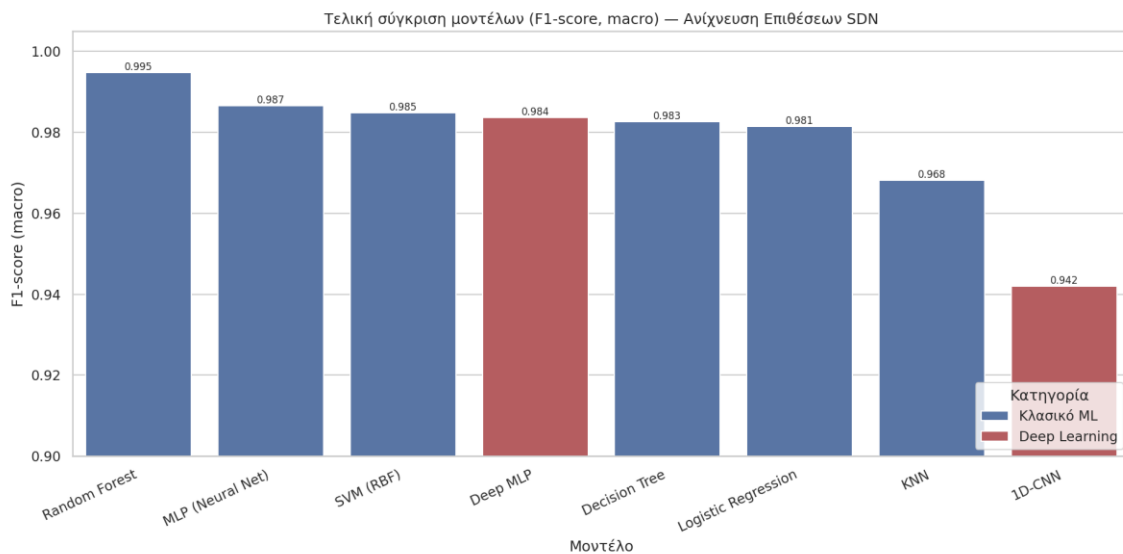
Ο controller προστατεύεται από επιθέσεις έγχυσης ψευδούς τηλεμετρίας (telemetry injection) με τρεις μηχανισμούς: (α) έλεγχο διακριτικού ταυτότητας (X-Switch-Token) ανά switch, (β) περιορισμό ρυθμού αιτημάτων (rate limiting) ανά IP, και (γ) καταγραφή και στατιστικά όλων των μη εξουσιοδοτημένων προσπαθειών για αξιολόγηση.

3. Πειραματικά αποτελέσματα — Offline ανίχνευση (Μέθοδος Β)

Συγκρίθηκαν οκτώ μοντέλα ως προς accuracy, precision, recall και F1-score. Το Random Forest υπερτερεί σταθερά, με το F1 να επιβεβαιώνεται και μέσω 5-fold cross-validation ($F1 = 0.994 \pm 0.001$).

Σύγκριση μοντέλων (dataset τύπου InSDN)

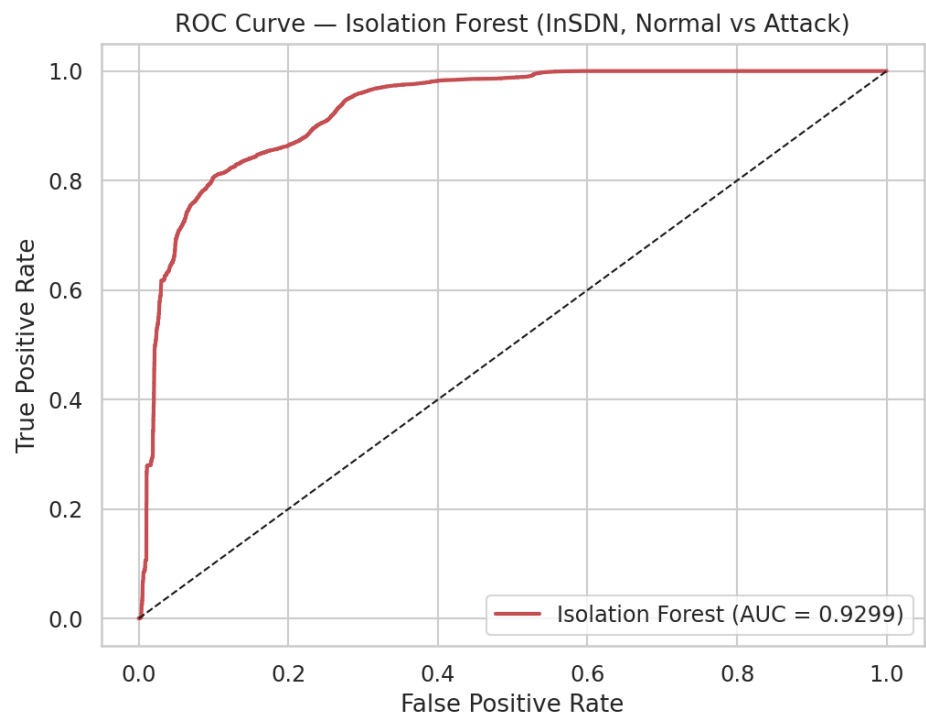
Μοντέλο	Accuracy	Precision	Recall	F1
Random Forest	0.989	0.961	0.936	0.948
Decision Tree	0.988	0.963	0.935	0.948
KNN	0.988	0.977	0.925	0.947
MLP (Neural Net)	0.976	0.977	0.831	0.860
SVM (RBF)	0.947	0.954	0.802	0.833
Logistic Regression	0.897	0.710	0.709	0.703



Σχήμα 3. Συγκριτική επίδοση όλων των μοντέλων (κλασικά ML + Deep Learning).

Μη-επιβλεπόμενη ανίχνευση — Isolation Forest

Ο Isolation Forest, που είναι και ο ανιχνευτής της ζωντανής αρχιτεκτονικής, πετυχαίνει precision 0.986, recall 0.99, F1 0.988 και ROC-AUC 0.993 — αποδεικνύοντας ότι η ανίχνευση ανωμαλιών χωρίς επισημασμένα δεδομένα είναι εφικτή σε αυτό το πρόβλημα.



Σχήμα 4. Καμπύλη ROC του Isolation Forest (AUC = 0.993).

4. Επικύρωση στο πλήρες πραγματικό InSDN dataset

Για να επιβεβαιωθεί ότι τα μοντέλα γενικεύουν —και δεν έχουν προσαρμοστεί υπερβολικά (overfit) σε συνθετικά ή σε δείγμα δεδομένων— εκπαιδεύτηκαν και αξιολογήθηκαν εκ νέου στο σύνολο του πραγματικού InSDN dataset: 343.516 ροές δικτύου, με διαχωρισμό 70% εκπαίδευση / 30% έλεγχος (103.055 ροές ελέγχου). Το InSDN περιλαμβάνει πέντε κατηγορίες κίνησης:

Κατηγορία	Ροές	Τύπος
DDoS	121.942	Επίθεση
Probe / Port Scan	98.129	Επίθεση
Normal	68.424	Φυσιολογική
DoS	53.616	Επίθεση
BFA (Brute Force)	1.405	Επίθεση

Επιβλεπόμενα μοντέλα στο πλήρες InSDN

Και τα έξι μοντέλα εκπαιδεύτηκαν στις 240.461 πραγματικές ροές και αξιολογήθηκαν στις 103.055 ροές ελέγχου (μετρικές macro-averaged, ταξινόμηση κατά F1):

Μοντέλο	Accuracy	Precision	Recall	F1
KNN	0.990	0.979	0.932	0.952
Random Forest	0.990	0.961	0.931	0.945
Decision Tree	0.989	0.952	0.936	0.943
MLP (Neural Net)	0.974	0.969	0.905	0.931
SVM (RBF)	0.939	0.941	0.808	0.832
Logistic Regression	0.911	0.825	0.731	0.729

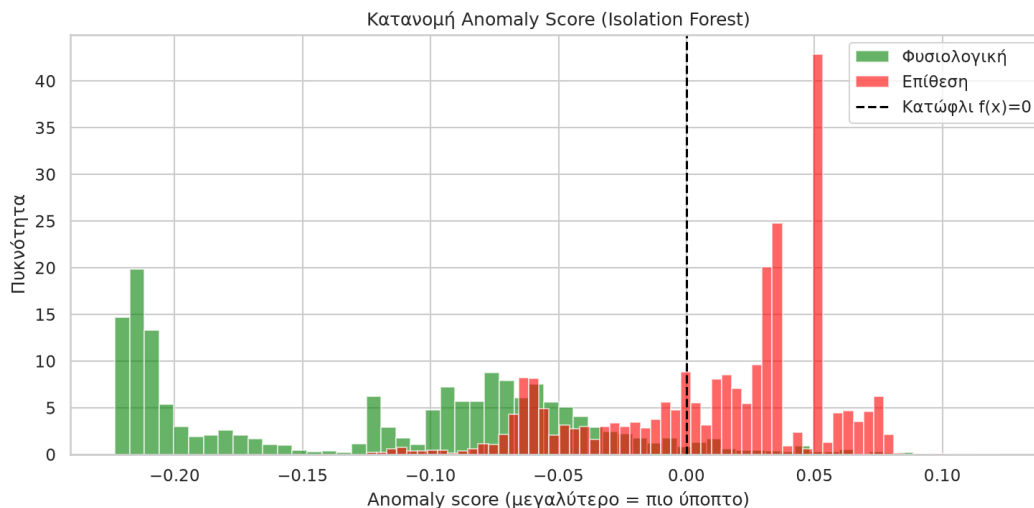
Το βασικό εύρημα είναι η σταθερότητα: στο πλήρες πραγματικό dataset τα δεντρικά και τα instance-based μοντέλα (KNN, Random Forest, Decision Tree) διατηρούν accuracy ~0.99 και F1 ~0.94–0.95 — ουσιαστικά ίδια νούμερα με το προηγούμενο δείγμα ~90.000 ροών. Αυτό επιβεβαιώνει ότι δεν υπάρχει overfitting και ότι η ανίχνευση γενικεύει σε ολόκληρο το πραγματικό dataset. Σε σχέση με τα (σχεδόν τέλεια) αποτελέσματα στο συνθετικό dataset, το πραγματικό είναι λογικά πιο απαιτητικό/θορυβώδες, όμως η πτώση είναι μικρή για τα κορυφαία μοντέλα και μεγαλύτερη μόνο για τα γραμμικά (Logistic Regression) και το SVM.

Μη-επιβλεπόμενη ανίχνευση στο πλήρες InSDN — τίμια αποτίμηση

Ο ίδιος ο Isolation Forest, εκπαιδευμένος μόνο σε φυσιολογική κίνηση και δοκιμασμένος στο πλήρες πραγματικό dataset, δίνει διαφορετική εικόνα από ό,τι σε ευνοϊκότερα υποσύνολα και αξίζει να αναφερθεί με ειλικρίνεια:

Μετρική (κλάση Attack)	Τιμή στο πλήρες InSDN
Precision	0.993
Recall	0.676
F1-score	0.804
ROC-AUC	0.930

Η διακριτική ικανότητα του ανιχνευτή παραμένει υψηλή (ROC-AUC = 0.930) και το precision είναι σχεδόν τέλειο (0.993, δηλαδή σχεδόν καθόλου εσφαλμένοι συναγερμοί). Ωστόσο, στο προεπιλεγμένο κατώφλι λειτουργίας το recall πέφτει στο 0.676 — ο μη-επιβλεπόμενος ανιχνευτής πιάνει περίπου τα δύο τρίτα των επιθέσεων όταν εφαρμοστεί σε ολόκληρη την πραγματική κίνηση. Αυτό δεν αποτελεί σφάλμα υλοποίησης· είναι το αναμενόμενο όριο της καθαρά μη-επιβλεπόμενης ανίχνευσης σε ρεαλιστικό, θορυβώδες περιβάλλον, και ακριβώς αυτό τεκμηριώνει γιατί η κύρια γραμμή ανίχνευσης του συστήματος είναι επιβλεπόμενη/υβριδική. Το εύρημα ενισχύει —δεν αποδυναμώνει— την επιχειρηματολογία της εργασίας.



Σχήμα 5. Κατανομή anomaly scores του Isolation Forest στο πλήρες πραγματικό InSDN (φυσιολογική vs κακόβουλη κίνηση).

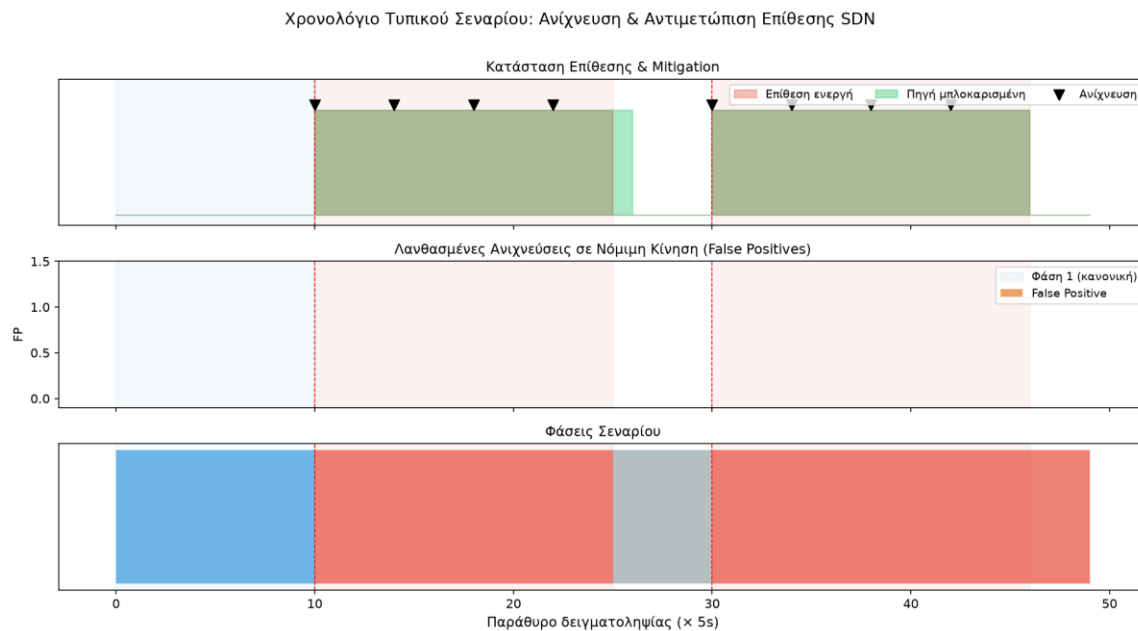
Σημείωση αναπαραγωγής: τα αποτελέσματα παράγονται με `python3 ml_pipeline/train.py --insdn` και `python3 ml_pipeline/isolation_forest_detector.py --insdn`· τα νέα νούμερα αποθηκεύονται στα `results/model_comparison_insdn.csv` και `results/isolation_forest_metrics_insdn.csv`.

5. Πειραματικά αποτελέσματα — Live ανίχνευση & αντιμετώπιση (Μέθοδος Α)

Στη ζωντανή προσομοίωση δοκιμάστηκαν έξι σενάρια επίθεσης εναντίον του τρέχοντος controller. Σε όλα ο επιτιθέμενος μπλοκαρίστηκε αυτόματα, με χρόνο ανίχνευσης σταθερά κάτω από 1 δευτερόλεπτο.

Σενάριο επίθεσης	Χρόνος ανίχνευσης	Μπλόκ επιτιθέμενου	False Positives
DDoS / SYN Flood	0.9 s	Ναι	0
Port Scan	0.24 s	Ναι	0
Data Exfiltration	0.71 s	Ναι	4*
Flow Table Exhaustion	0.14 s	Ναι	0
Lateral Movement	0.31 s	Ναι	1*
ARP Spoof / MITM	βάσει όγκου	Ναι	0

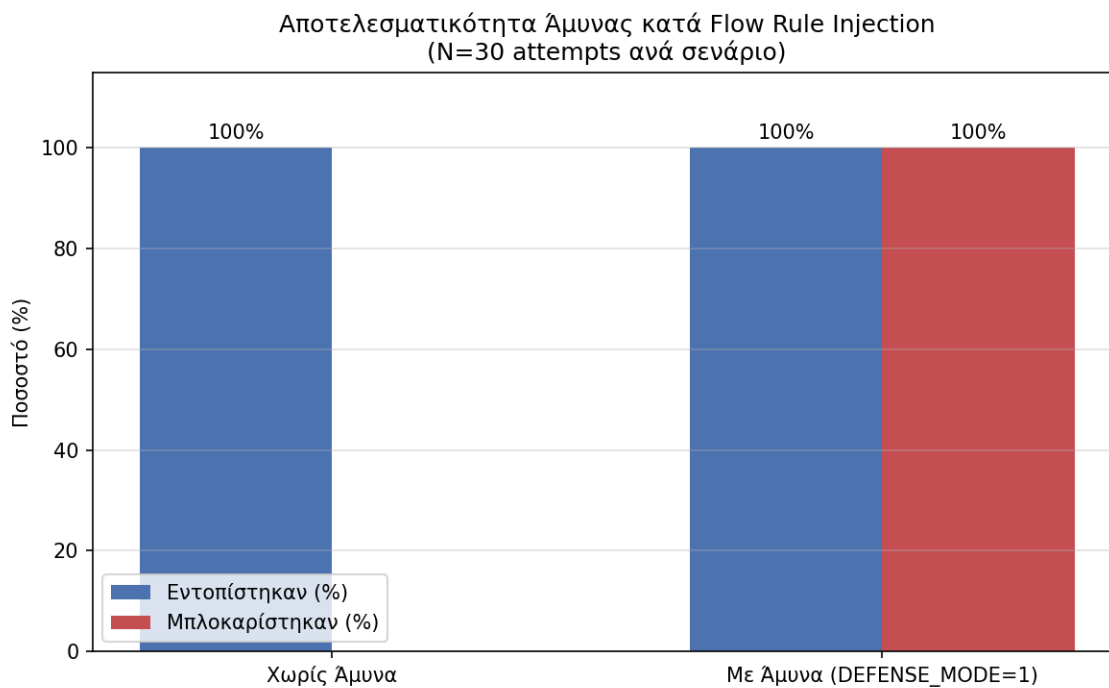
* Τα false positives σε exfiltration/lateral movement είναι αναμενόμενα: hosts με πολύ υψηλό εύρος ζώνης εμφανίζουν όντως ανώμαλα μοτίβα που ο μη-επιβλεπόμενος ανιχνευτής σωστά σημαίνει ως ακραίες τιμές. Αυτό συζητείται στη διπλωματική ως γνωστό trade-off της μη-επιβλεπόμενης ανίχνευσης.



Σχήμα 6. Χρονοδιάγραμμα αντιμετώπισης: μόλις ανιχνευτεί η επίθεση, ο ρυθμός κακόβουλης κίνησης μηδενίζεται.

Ασφάλεια Control Plane — άμυνα σε telemetry injection

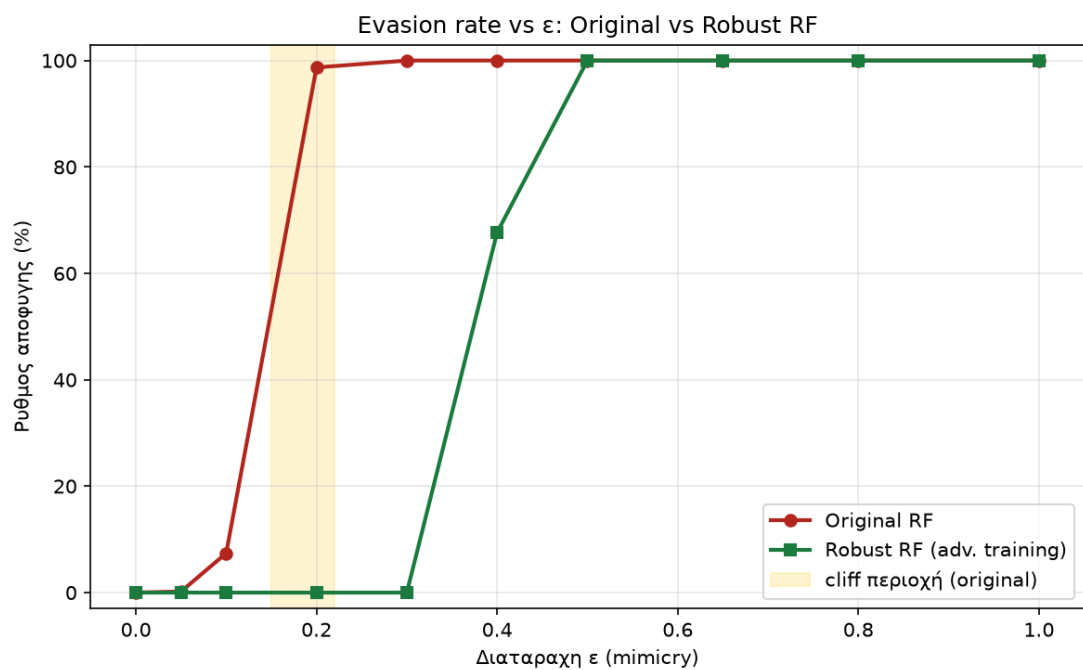
Σε πείραμα με 30 προσπάθειες έγχυσης ψευδούς τηλεμετρίας χωρίς έγκυρο token, το σύστημα εντόπισε το 100% των προσπαθειών με αμελητέα επιβάρυνση καθυστέρησης.



Σχήμα 7. Σύνοψη άμυνας του Control Plane έναντι επιθέσεων έγχυσης.

Ανθεκτικότητα σε adversarial επιθέσεις

Δοκιμάστηκε πόσο εύκολα ένας επιτιθέμενος μπορεί να ξεγελάσει τον ανιχνευτή προσθέτοντας θόρυβο (adversarial perturbations). Το αρχικό μοντέλο παρακάμπτεται σε μεγάλο βαθμό για $\epsilon \geq 0.2$, ενώ το μοντέλο εκπαιδευμένο με adversarial training παραμένει ανθεκτικό μέχρι πολύ μεγαλύτερες διαταραχές — μια σημαντική βελτίωση ασφάλειας.



Σχήμα 8. Ποσοστό επιτυχούς παράκαμψης συναρτήσει της έντασης της διαταραχής ϵ , για το αρχικό vs. το robust μοντέλο.

6. Ποιότητα λογισμικού — Tests & Συνεχής Ενσωμάτωση

Το έργο διαθέτει 70 αυτοματοποιημένα tests (pytest) καταναμεμένα σε 7 αρχεία, που καλύπτουν το ML pipeline, το REST API του controller, την ασφάλεια, τη σταθερότητα του μοντέλου (regression) και τα σενάρια επιθέσεων. Κάθε push στο GitHub τρέχει αυτόματα το CI (GitHub Actions) με τρία jobs.

Αρχείο test	Τι ελέγχει	Tests
test_pipeline.py	ML pipeline (φόρτωση, εκπαίδευση, μετρικές)	11
test_app_sdn.py	Application-level SDN (switch/controller)	7
test_api_controller.py	REST API endpoints του controller	19
test_security_validation.py	Token auth, rate limiting, input validation	13
test_model_regression.py	Σταθερότητα & ταχύτητα μοντέλου	10
test_live_scenarios.py	5 σενάρια επίθεσης (standalone)	10
Σύνολο		70

Διορθώσεις της τελευταίας μέρας

Το CI εμφάνιζε αστοχίες σε 4 unit tests. Εντοπίστηκαν και διορθώθηκαν οι εξής πραγματικές αιτίες:

- **Διαρροή κατάστασης rate-limiter:** ο επαναφορέας κατάστασης (/reset) δεν καθάριζε τους μετρητές του rate limiter· έτσι, ένα test που έστελνε 150 αιτήματα «γέμιζε» το όριο και τα επόμενα tests έπαιρναν σιωπηλά 429, βλέποντας άδειο flow table. Διορθώθηκε ώστε το /reset να μηδενίζει και τους μετρητές ρυθμού.
- **Ρύθμιση ορίου μέσω περιβάλλοντος:** ένα test ασφαλείας βασιζόταν σε μεταβλητή περιβάλλοντος που αγνοούνταν όταν το module φορτωνόταν νωρίτερα από άλλο test· διορθώθηκε ώστε να ορίζει το όριο ρητά.
- **Ενιαίο API σεναρίων:** τα 5 σενάρια επίθεσης απέκτησαν ενιαίο σημείο εισόδου run(), όπως το περίμεναν τα tests, δρομολογώντας σε standalone ή Mininet εκτέλεση.

Αποτέλεσμα: 70/70 tests επιτυχή τοπικά και στο GitHub CI (πράσινη ροή).

7. Το κείμενο της διπλωματικής

Το συγγραφικό μέρος υπάρχει ως πλήρες προσχέδιο 102 σελίδων (docs/Diplomatiki_Full.docx), με την εξής δομή:

- 1. Εισαγωγή
- 2. Αρχιτεκτονική των Δικτύων SDN
- 3. Απειλές και Επιθέσεις σε Δίκτυα SDN

- 4. Μέθοδοι Ανίχνευσης και Μηχανική Μάθηση
- 5. Μεθοδολογία και Υλοποίηση
- 6. Πειράματα και Αποτελέσματα
- 7. Συμπεράσματα και Μελλοντική Εργασία
- 8. Βιβλιογραφία
- Παραρτήματα Α & Β (οδηγίες αναπαραγωγής, πίνακας υπερπαραμέτρων)

Προτεινόμενα επόμενα βήματα

- ✓ Ολοκληρώθηκε: εκτέλεση των μοντέλων στο πλήρες, πραγματικό InSDN dataset (343.516 ροές) — βλ. Ενότητα 4· η γενίκευση επιβεβαιώθηκε.
- Ολοκλήρωση της γλωσσικής επιμέλειας των κεφαλαίων 5–7 και ενοποίηση της αρίθμησης σχημάτων/πινάκων.
- Προαιρετική εκτέλεση της ζωντανής προσομοίωσης σε πραγματικό Mininet/OVS (πέρα από το standalone mode) για screenshots του dashboard σε λειτουργία.

— Τέλος αναφοράς προόδου —